

Data Engineering — All Past Content

Collected from the series content already created in this project. Core DE + ADF content delivered so far.

Day 1 — What is Data Engineering?

Jab bhi aap Swiggy pe order karte ho...

order place hota hai...

payment hoti hai...

delivery status update hota hai...

yeh poora process data generate karta hai.

But yeh data...

store kahan hota hai?

Kaise move hota hai?

Aur jab company ko is data se business insights nikalni hoti hain...

yeh data ready kaun karta hai?

Yahin pe aata hai...

Data Engineering.

Simple language mein...

Data Engineering ka kaam hai data ko source se lekar... business ke liye usable form tak pahunchana.

For example...

data aa sakta hai database se... API se... CSV files se... ya application events se.

Ab is data ko hum ingest karte hain...

phir store karte hain...

clean aur transform karte hain...

aur finally analytics ya reporting ke liye ready karte hain.

Flow:

Source → Ingestion → Storage → Transformation → Analytics

Data Engineer ka kaam sirf data move karna nahi hai.

Reliable, usable aur scalable data system build karna hai.

Ye data aata kahan se hai, aur source database mein actually kis type ka workload hota hai?

Wahin se... OLTP vs OLAP.

Day 2 — OLTP vs OLAP

Abhi Day 1 mein humne dekha tha...

Data Engineer ka actual kaam kya hota hai.

But ab ek basic question hai...

Ye data aata kahan se hai?

Suppose aap Amazon pe ek order place karte ho.

Order create hua...

payment hui...

inventory update hui...

order status bhi update ho gaya.

Ye day-to-day transactions hain.

OLTP — Online Transaction Processing.

Main focus: bahut saari small transactions quickly.

Insert/update/delete.

Now business asks: last 2 years mein kis product ne sabse zyada revenue generate kiya?
That is large historical analysis.
OLAP — Online Analytical Processing.

Focus: large data, read, multiple tables, aggregation, business insights.

Example:

Order placed = OLTP.

Last 12 months total sales = OLAP.

Flow:

Application → OLTP Database → Data Pipeline → OLAP/Data Warehouse → BI/Analytics.

Simple rule:

Small, frequent transactions → OLTP.

Large, historical analysis → OLAP.

OLTP se data OLAP tak jaata kaise hai?

ETL vs ELT.

Day 3 — ETL vs ELT

Abhi humne dekha...

OLTP mein day-to-day transactions hote hain...

aur OLAP mein hum us data ko analyse karte hain.

But ek question...

OLTP ka data OLAP tak jaata kaise hai?

E-commerce order data OLTP mein hai, aur humein Data Warehouse mein chahiye.

Source data mein cleaning/transformation ki zarurat ho sakti hai:

missing values, different formats, duplicates.

ETL = Extract → Transform → Load.

Pehle extract, transform, then load.

ELT = Extract → Load → Transform.

Pehle target warehouse/lakehouse mein load, phir transform.

Modern cloud platforms mein ELT common hai because they provide strong compute.

But koi universal winner nahi hai — architecture, volume, source aur target decide karte hain.

Simple rule:

ETL → Transform before Load.

ELT → Transform after Load.

Ye analytical data actually store kahan karenge?

Data Lake? Data Warehouse? Ya kuch aur?

Data Lake vs Data Warehouse vs Lakehouse.

Day 4 — Data Lake vs Data Warehouse vs Lakehouse

Abhi humne samjha... OLTP mein day-to-day transactions hote hain, aur OLAP mein hum data analyse karte hain.

But ek question... OLAP ke liye data rakhenge kahan?

E-commerce company mein orders SQL DB mein, CRM customer data, APIs se product data, aur website logs JSON mein ho sakte hain.

Data Lake = large raw, flexible storage. Structured, semi-structured aur unstructured data store kar sakte ho.

Problem: raw data analytics ke liye messy ho sakta hai.

Data Warehouse = cleaned, structured, analytics-ready data.

Lakehouse = Data Lake ki flexibility + Data Warehouse ki reliable table and analytics capabilities.

Simple rule:

Raw/flexible → Data Lake.

Structured business analytics → Data Warehouse.

Dono capabilities chahiye → Lakehouse.

Lakehouse ke andar data ko organize kaise karte hain?

Medallion Architecture.

Day 5 — Medallion Architecture

Lakehouse mein data aa gaya... ab organize kaise karenge?

Orders, customers, products aur logs agar raw form mein pade rahe... reporting messy ho jayegi.

Medallion Architecture:

Bronze → Silver → Gold.

Bronze = raw/source-like data, preserve the source.

Silver = clean and standardized data: duplicates, nulls, data types, standardization.

Gold = business-ready data: sales summary, customer analytics, revenue metrics.

Flow:

Source → Bronze → Silver → Gold → BI.

Simple rule:

Bronze = Preserve.

Silver = Trustworthy.

Gold = Business-ready.

Bronze aur Silver mein data store kis format mein hoga?

Parquet vs Delta Lake.

Day 6 — Parquet vs Delta Lake

Parquet ek file format hai. Ye columnar storage use karta hai, isliye analytics ke liye efficient hota hai.

But Parquet alone database-like update/delete/merge capabilities nahi deta.

Delta Lake Parquet ke upar reliable table capabilities provide karta hai:

ACID transactions, schema enforcement/evolution, UPDATE, DELETE, MERGE aur time travel.

Example: Order 101 existing hai aur status update hua. Delta MERGE se existing record update kar sakte ho, new record ho toh insert.

Simple rule:

Parquet = how data is stored.

Delta Lake = reliable table capabilities on top.

Agar source mein sirf 10,000 records change hue, kya poora data dobara load karenge?

Full Load vs Incremental Load.

Day 7 — Full Load vs Incremental Load

Suppose 10 crore records hain aur daily sirf 20,000 records change hote hain.

Har run mein 10 crore load karna = Full Load. Simple, but expensive for large production tables.

Incremental Load = sirf new ya changed records process karna.

Full load useful ho sakta hai initial load ya small datasets ke liye.
Incremental load large production data ke liye generally better hota hai.

Simple rule:

Full Load → Everything.

Incremental Load → Only new or changed data.

Pipeline ko pata kaise chalega last run ke baad ka data kaunsa hai?

Watermark.

Day 8 — Watermark-Based Incremental Load

Suppose source table mein updated_at hai.

Last successful run ka watermark 10:00 AM tha.

Next run:

updated_at > 10:00 AM

Sirf uske baad updated records process karo.

Latest successfully processed time 11:30 AM ho toh next watermark 11:30 AM.

Important: processing successful hone se pehle watermark update mat karo. Failure ke case mein records skip ho sakte hain.

Simple rule:

Watermark = last successfully processed position.

Reliable updated_at nahi hai, aur insert/update/delete actual changes capture karne hain?
CDC.

Day 9 — Change Data Capture (CDC)

CDC — Change Data Capture — source mein hone wale actual changes capture karta hai.

Customer city Pune se Mumbai = UPDATE.

New customer = INSERT.

Customer deleted = DELETE.

Watermark aur CDC ka simple difference:

Watermark ek position track karta hai.

CDC actual changes track karta hai.

Reliable timestamp aur simple requirement ho toh watermark enough ho sakta hai.

Insert/update/delete accurately capture karne hain toh CDC useful hai.

Simple rule:

Watermark → Where to continue.

CDC → What changed.

Pipeline fail ho gayi aur retry ki... duplicate data kaise avoid karoge?

Idempotency.

Day 10 — Idempotency + Duplicate + Late-Arriving Data

Pipeline ko 10,000 changed records process karne hain. 7,000 process hue, phir failure.

Retry par same records dobara aa sakte hain.

Idempotency ka matlab:

same input ko one or multiple times process karo... final result same hona chahiye.

Unique keys, deduplication, MERGE/upsert jaise techniques help kar sakti hain.

Late-arriving data:

event time 10:05 AM hai, but data 10:30 AM par aaya.

Event time aur ingestion time ko carefully handle karna padta hai.

Simple rule:

Good production pipeline should be safe to retry and should handle late data without corrupting results.

Ab data modeling side pe:

Normalization vs Denormalization.

Day 11 — Normalization vs Denormalization

Suppose ek customer ne 50 orders place kiye. Agar har order mein customer ka full name, city, email repeat karoge, redundancy badhegi.

Normalization mein customer information separate table mein rakho, Orders mein Customer_ID rakho.

Goal: less redundancy and better consistency.

Denormalization mein related information combine ya intentionally duplicate ki ja sakti hai, so analytics queries simple ho aur joins kam hon.

Simple rule:

Normalization → reduce redundancy and improve consistency.

Denormalization → simpler analytics and fewer joins.

Analytics table design mein Fact aur Dimension ka role kya hai?

Fact Table vs Dimension Table.

Day 12 — Fact Table vs Dimension Table + Star Schema

Fact table tells you what happened and contains measurable values:

Order ID, Customer ID, Product ID, Date ID, Quantity, Sales Amount, Discount.

Dimension tables contain descriptive information:

customer name/city/segment, product name/category/brand, date month/quarter/year.

Fact = event + measures.

Dimension = description.

Central Fact Table ke around Customer, Product, Date jaise dimensions connect hote hain — this is Star Schema.

Simple rule:

Fact tells you what happened.

Dimension tells you about the thing.

Star Schema aur Snowflake Schema mein difference kya hai?

Aur history maintain kaise karenge?

Star vs Snowflake + SCD.

Day 13 — Star Schema vs Snowflake Schema + SCD Type 1 & 2

Star Schema mein dimensions Fact Table se directly connected hoti hain.

Snowflake Schema mein dimensions further normalized hoti hain:

Fact Sales → Product → Category → Department.

Star simpler and fewer joins; Snowflake can reduce redundancy but adds joins.

SCD example:

Customer city Pune se Mumbai.

Type 1: old value overwrite — history not required.

Type 2: old version preserve, new version create — history required, usually dates/current flag ke saath.

Simple rule:

History nahi chahiye → Type 1.

History chahiye → Type 2.

Warehouse design mein sabse pehle kya decide karna chahiye?

Grain.

Day 14 — Data Warehouse Design + Grain

Warehouse design ka basic question:

One row actually represents what?

Isko Grain kehte hain.

Order mein 3 products hain.

Order grain = one row per order.

Order-line grain = 3 rows, one for each product.

Grain clear nahi hua toh revenue aur quantity double count ho sakte hain.

Isliye fact table design se pehle grain define karo.

Then identify facts, dimensions, keys, relationships and required history.

Simple rule:

Grain means: one row represents what?

Next:

Data Partitioning.

Day 17 — Data Partitioning

Suppose table mein 10 crore records hain, but query sirf March 2026 ka data chah rahi hai.

Agar poore 10 crore scan hue toh unnecessary data read hoga.

Partitioning se data month/year jaise logical chunks mein divide kar sakte hain:

2026-01, 2026-02, 2026-03.

March query relevant partition ko target kar sakti hai, reducing unnecessary reads.

But partition key carefully choose karo. Customer ID jaise high-cardinality column se bahut tiny partitions ban sakte hain.

Simple rule:

Partitioning ka goal data ko bas divide karna nahi...

unnecessary data reading reduce karna hai.

Source schema change ho jaye toh?

Schema Evolution vs Schema Drift.

Day 18 — Schema Evolution vs Schema Drift + Data Contracts

Customer file pehle id, name, city tha. Planned way mein email add hua — Schema Evolution.

Unexpectedly city ka naam change ho gaya, integer string ban gaya, ya required column remove ho gaya — Schema Drift.

Simple:

Evolution = expected/controlled change.

Drift = unexpected change.

Data Contract producer aur consumer ke beech agreed structure hota hai: fields, types, required fields, format and quality expectations.

Simple rule:

Evolution = controlled change.

Drift = unexpected change.

Contract = agreed structure and expectations.

Schema correct hone ke baad bhi data wrong ho sakta hai.

Data Quality vs Data Governance.

Day 19 — Data Quality vs Data Governance

Pipeline successfully complete ho sakti hai but data wrong ho sakta hai.

Expected 10 lakh orders, warehouse mein 8 lakh — data quality problem.

Quality checks:

nulls, duplicates, negative sales, row counts, freshness, invalid values.

Governance asks:

owner kaun hai, access kis user ko hai, sensitive data hai ya nahi, lineage kya hai, catalog available hai ya nahi.

Simple difference:

Data Quality asks: Is the data correct and trustworthy?

Data Governance asks: Is the data properly managed, controlled and governed?

Business ko sirf data nahi chahiye... trusted data chahiye.

Trusted platform reliable kaise banega?

Reliability, Monitoring and complete architecture.

Day 20 — Data Reliability + Monitoring + End-to-End DE Architecture

Production pipeline mein failure ho sakta hai:

source unavailable, late data, schema change, duplicates, processing failure.

Data Reliability ke liye retry, error handling, idempotency, data quality checks, logging, monitoring and alerts important hain.

Complete flow:

Sources → Ingestion → Storage → Bronze → Silver → Gold → Data Modeling → Analytics / BI.

Supporting concepts:

incremental loading, CDC, partitioning, data quality, governance, monitoring and reliability.

Simple rule:

Data Engineering is not just about moving data.

It is about building scalable, reliable and trustworthy data systems.

Aur yehi Data Engineer ki real responsibility hai.

Azure Data Factory — Series Content

ADF content delivered so far. Day 4 is not included because it had not yet been delivered at the time of this PDF request.

ADF Day 1 — What is Azure Data Factory?

Data Engineering mein ek common problem hai...
data ek jagah nahi hota.

Maan lo customer data SQL Server mein hai...
orders CSV files mein hain...
aur kuch data REST API se aa raha hai.

Ab in sab data sources ko ek central data platform tak regularly kaise laoge?
Manually? Obviously nahi.

Yahin pe aata hai... Azure Data Factory — ADF.

ADF ek cloud-based data integration and orchestration service hai.

Matlab ADF different data sources se data ko connect kar sakta hai...
move kar sakta hai...
aur data workflow ko schedule aur orchestrate kar sakta hai.

Example:

SQL Server daily orders → ADLS.

ADF pipeline is process ko automate kar sakti hai.

ADF scheduling, dependencies, failure handling and monitoring mein bhi help karta hai.

Important:

ADF har type ka heavy data processing engine nahi hai.

Heavy transformation Databricks, Synapse, SQL, Fabric etc. kar sakte hain, aur ADF un workflows ko orchestrate kar sakta hai.

Simple rule:

ADF is the control center of your data pipeline.

It connects your sources, moves the data, and coordinates what happens next.

ADF ke andar kaun-kaun se components milke ye pipeline banate hain?

ADF Main Components.

ADF Day 2 — ADF Main Components

Kal humne samjha... ADF actually karta kya hai.

But jab ADF open karoge...

Pipeline, Dataset, Linked Service, Activity?

Confusion ho sakta hai.

Suppose SQL Server se customer data ADLS mein jaana hai.

Pipeline = complete workflow. Ye sequence define karti hai.

Activity = individual task.

Copy, Lookup aur other activities.

Linked Service = ADF ko batata hai external service se connect kaise karna hai.

Dataset = actual data reference.

Kaunsi table, file ya path use karna hai.

Integration Runtime = data integration activities ke liye connectivity and compute infrastructure.

Simple flow:

Pipeline → Activity → Dataset → Linked Service

with Integration Runtime providing the required connectivity/execution environment.

Simple rule:

Pipeline = workflow

Activity = task

Linked Service = connection

Dataset = data reference

Integration Runtime = connectivity/execution infrastructure

Linked Service exactly connection kaise define karta hai?

Linked Services.

ADF Day 3 — Linked Services

ADF ke main components samjhe...

Pipeline, Activity, Dataset...

But ADF ko pata kaise chalega data actually kahan hai?

Suppose SQL Server.

ADF ko server/address, database and authentication details chahiye.

Yahin aata hai Linked Service.

Simple language:

Linked Service is the connection information for an external service.

Example:

ADF → Linked Service → SQL Server.

ADLS ke liye bhi Linked Service create kar sakte ho.

Common confusion:

Linked Service vs Dataset.

Linked Service = kis service se connect karna hai aur how.

Dataset = us service ke andar kaunsa data work karna hai.

Example:

SQL Server Linked Service.

Customers table ko Dataset represent kar sakta hai.

Simple analogy:

Linked Service = building address.

Dataset = specific room.

Production mein credentials hardcode karne ke instead Managed Identity, Key Vault and secure authentication use ki ja sakti hai.

Simple rule:

Linked Service tells ADF how to connect.

Dataset tells ADF what data to work with.

Connection define ho gaya...

but ye connection/data movement actually execute/connect kaise hoga?

Integration Runtime.